Testing routines.sql locally

CROSSFADE, info579_group_1 (Herling, Nathan; Pabari, Nidhi Nilesh). INFO 579 Project 2, Task 4. 50 tests; last run 2026-11-08 13:24:10: 50 passed, 0 failed.

How routines.sql is checked: MySQL's own catalogue must hold each routine exactly as its comment declares it; each routine is called on the crossfade database and must write, return and refuse exactly what an independent Python answer from data.csv says; the RA's rules (no derived values as input, a royalty computed once, plays in UTC) are tested directly; and the duplicate-upload handler is pushed to its edges: a duplicate answered, every other error passed on, a key clash re-raised.

1. What the tests compare against

Source	From	How it is used
Independent answers	oracle_routines.py + data.csv	What each routine must write, return or refuse, worked out in plain Python from data.csv and the RA's rules, no SQL. MySQL must agree value for value, including the error code of every refusal.
MySQL's catalogue	information_schema.ROUTINES, PARAMETERS	Each routine's kind, parameters, return type and data-access characteristic, as MySQL stored them, compared with the comment.
The RA's integrity rules	requirements_analysis.pdf	I7 (completed and royalty computed by the procedure that records a play, once, never updated), I6 (shared stations), the play key that keeps a double-counted royalty out of the ledger, and "the procedures do not accept derived values as input".
Live behaviour	rolled-back calls, two connections, a test trigger	Every call runs in a transaction that is rolled back; the race uses two connections; the RESIGNAL branch is forced with a temporary trigger, dropped afterwards.
The brief and the slides	info579_project02.pdf, Unit 9	2 procedures + 1 function or 1 + 2, in one file; DELIMITER, DECLARE, IF, SIGNAL, handlers and IN / OUT parameters from Unit 9.

2. Run them

a. What you need. A running MySQL Server 8.0.16 or later and its root password; Python 3.10 or later. --fresh builds the database; the tests create the routines themselves by running routines.sql.

b. Get the files. Download crossfade_routines_tests.zip with the button above and extract it. It holds one folder, crossfade_routines_tests, with the tests, the oracle, routines.sql, database_implementation.sql, data.csv, use_cases.json and the guide.

```
Expand-Archive -Force $HOME\Downloads\crossfade_routines_tests.zip .
cd .\crossfade_routines_tests
```

c. Set up Python (once). A virtual environment with the three packages. macOS / Linux: python3 and .venv/bin/python.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
```

d. Run the tests. --fresh rebuilds the crossfade database from database_implementation.sql first. The tests then run routines.sql (it drops and re-creates the three routines) and call each one; every write is rolled back, and the race and key-clash cases delete what they add.

```
$env:CROSSFADE_DB_PASSWORD = "your-root-password"
.venv\Scripts\python -m pytest --fresh
```

e. Read the result. One line per test, then a summary. A failure names the comment field, catalogue entry, column, value or error code that differs from the oracle.

```
collected 50 items
test_routines.py::test_comment_contract[F1] PASSED   [  2%]
...
========== 50 passed in 1.0s ==========
```

3. What each group proves

Group	Tests	What it proves
contract	18/18	Each routine is preceded by a purpose comment with its tag and name, use cases, business need, what it does step by step, the requirement, the Unit 9 features it uses, its tables, parameters, return type, errors and an example call. MySQL's own catalogue (information_schema.ROUTINES and PARAMETERS) holds it exactly as declared: its kind, every parameter in order with mode, name and type, and a function's return type. The characteristic tells the truth (a function that reads tables is READS SQL DATA and NOT DETERMINISTIC; a procedure that writes is MODIFIES SQL DATA). Every feature listed under Uses: is in the body. It names exactly the tables listed, counting those of a routine it calls, and every column it touches is declared by one of its use cases.
behaviour	24/24	Each routine is called on the crossfade database and compared with an independent answer computed in plain Python from data.csv (oracle_routines.py): the row a procedure writes, its OUT values, a function's result, and the refusals it must give, both its own SIGNALs and the database's errors it must not swallow. Every call runs inside a transaction that is rolled back. Two cases go further: two connections upload at the same moment, and a temporary trigger forces a primary-key clash so the handler's RESIGNAL branch runs.
coverage	8/8	The file as a whole: the brief's mix (2 procedures + 1 function, or 1 + 2); tags F1.., P1.. in order; every use case with routine columns served, and every declared column touched by a routine serving it; the RA's rule that procedures take no derived values as input; no transaction control inside a routine; and the script itself: it re-creates the routines when run again, and its Try it block leaves the data unchanged and returns what its comments promise.

4. Files

File	Role
routines.sql	the file under test (Task 4)
test_routines.py	the tests: contract, behaviour cases, file coverage
oracle_routines.py	the independent answers: what each routine must write, return or refuse, in plain Python from data.csv
conftest.py	shared set-up: finds the files, connects, splits routines.sql at its DELIMITER lines, writes the results snapshot
use_cases.json	the use cases: their RA evidence and the columns each routine may read or write (routineColumns)
data.csv	the Project 1 sample data the oracle reads
database_implementation.sql	builds the database the routines run on (--fresh)
pytest.ini	pytest settings (verbose, no cache folder)
requirements.txt	pytest, PyMySQL, cryptography (MySQL 8 sign-in)
test_routines_guide.pdf	how to run the tests locally, what they cover and what they could miss
mutate_routines.py	tests the tests: breaks routines.sql 20 ways, one at a time, and checks every break is caught (repo only)
crossfade_routines_tests.zip	the tests, the oracle, every input file and the guide, in one folder that runs as-is

5. What the tests could miss

Passing means the file agrees with its sources and refuses what they forbid. It does not mean the following:

Gap	What could slip through	What covers it instead
Independent code, not an independent reading	The oracle was written from the same comments and RA rules as the routines. If a rule was misread (what 'released' means, whether 30 seconds exactly qualifies), the routine and its expected answer agree.	Each comment quotes the RA rule it applies; a second reader.
Columns found by name, not by plan	EXPLAIN cannot see inside a routine, so the column check counts the column names that appear in its body. A name shared by two tables it uses (track_id, artist_id) counts for both.	The tables are checked separately; the behaviour cases check what is actually written.

Gap	What could slip through	What covers it instead
One interleaving of a race	The simultaneous-upload case runs one timing: A uploads, B waits, A commits. On MySQL 8 the second upload's ID lookup waits for the first, so B takes the next ID; other timings, and other servers, are not tried.	The handler's RESIGNAL branch is tested directly with a forced key clash.
A forced clash, not a natural one	The primary-key clash that RESIGNAL guards against is produced by a temporary trigger, because it does not happen on its own here. The test proves the branch works, not how often it is needed.	The comment says the branch is defensive.
Permissions not tested	The RA grants the application EXECUTE on the procedures and no direct writes, making them the only write path. The tests run as root and do not check grants (they are not part of the brief).	Not required by the brief.
One time zone	"Recorded in UTC" is tested with the session set to UTC-7 (Arizona). Other offsets and daylight-saving changes are not tried.	UTC_TIMESTAMP ignores the session time zone by definition.
Comments checked for shape, not truth	Every field must be present and agree with MySQL (kind, parameters, tables, features). Whether the business reasoning is sound is not testable.	The comments cite the RA.

6. Testing the tests

mutate_routines.py copies the file, makes one deliberate mistake, and runs every test against the copy. A mistake is caught when a test fails that passes on the real file. 20 of 20 were caught.

Mistake	Caught	First failing tests
a play of exactly 30 seconds earns nothing (> instead of >=)	yes (3 failed)	test_f1_thirty_seconds_is_the_boundary, test_f1_follows_a_rate_change, test_try_it_block_gives_the_documented_answers
a recording that does not exist earns 0 instead of NULL	yes (1 failed)	test_f1_unknown_recording_has_no_royalty
every recording paid at one artist's rate (join condition wrong)	yes (3 failed)	test_f1_agrees_with_every_recorded_play, test_p1_full_length_play_is_completed, test_p1_shared_station_is_accepted
declared DETERMINISTIC, though the rate it reads can change	yes (2 failed)	test_data_access_declared[F1], test_uses_what_it_lists[F1]
declared NO SQL, though it reads two tables	yes (2 failed)	test_data_access_declared[F1], test_uses_what_it_lists[F1]
completed means 30 seconds or more, not the whole recording	yes (3 failed)	test_p1_records_an_abandoned_skip, test_p1_ledger_survives_a_rate_change, test_try_it_block_gives_the_documented_answers
only rejected recordings refused: one awaiting scoring can be streamed	yes (1 failed)	test_p1_refuses_an_invalid_play[awaiting_scoring]
a play as long as the recording refused (>= instead of >)	yes (3 failed)	test_p1_full_length_play_is_completed, test_p1_shared_station_is_accepted, test_p1_repeated_report_is_rejected
a fixed royalty instead of the artist's rate through play_royalty	yes (5 failed)	test_reads_declared_tables[P1], test_p1_full_length_play_is_completed, test_p1_short_play_earns_nothing
no time given: local time (NOW) instead of UTC	yes (1 failed)	test_p1_no_time_given_records_utc_now
the station that surfaced the play not recorded	yes (1 failed)	test_p1_shared_station_is_accepted
a repeated report of a play silently ignored (CONTINUE HANDLER FOR 1062)	yes (1 failed)	test_p1_repeated_report_is_rejected
the comment lists a feature the procedure does not use	yes (1 failed)	test_uses_what_it_lists[P1]
the comment declares a parameter type MySQL does not have	yes (1 failed)	test_created_as_declared[P1]

Mistake	Caught	First failing tests
the handler catches every error, not just 1062	yes (2 failed)	test_uses_what_it_lists[P2], test_p2_handler_catches_only_duplicates
a key clash on the new track ID reported as a duplicate (RESIGNAL removed)	yes (2 failed)	test_uses_what_it_lists[P2], test_p2_key_clash_is_not_reported_as_a_duplicate
an upload enters the catalogue as scored, skipping the pipeline	yes (1 failed)	test_p2_new_upload_awaits_scoring
the new track takes the highest existing ID instead of the next one	yes (35 failed)	test_comment_contract[F1], test_comment_contract[P1], test_comment_contract[P2]
an unregistered artist is not refused in words (left to the foreign key)	yes (3 failed)	test_reads_declared_tables[P2], test_p2_unregistered_artist_is_refused, test_declared_columns_are_touched
artist left off the comment's Tables line	yes (1 failed)	test_reads_declared_tables[P2]

7. Findings

The deliberate-mistake run had a blind spot of its own (now fixed). Its first run reported one mistake missed: catching every error instead of only 1062. The edit had landed in the file's header comment, where the same words appear, so the routine never changed. The edit now names the handler's own code line, and that mistake is caught. The other 19 edits were checked by hand to land where they mean to: 16 in code, 3 in the comment fields the contract tests read.

Simultaneous uploads wait for each other on MySQL 8. The two-connection case showed that the second upload's lookup of the next track ID waits for the first upload to commit, then takes the next ID. The primary-key clash the handler re-raises (RESIGNAL) therefore does not occur on its own here; the branch is tested with a forced clash.